



American International University-Bangladesh (AIUB)
Department of Computer Science Faculty of Science & Technology (FST)

PROJECT TITLE

A Software Engineering Project Submitted

By

Semester : Summer 24-25		Section: F	Group Number: 08	
SL	Student Name	Student ID	Contribution (CO3 + CO4)	Individual Marks
1	Md. Minhaj Rowfun Rabbi Anik	23-54110-3		
2	Argha Ghosh	23-50804-1		
3	Samiul Islam	23-51004-1		
4	Tahomina tafsir	23-50229-1		
5	Md. Jonayed Hossain	23-50157-1		

The project will be evaluated for the following Course Outcomes

CO3 (PO-g-1) <i>Select appropriate software engineering models, project management roles and their associated skills for the complex software engineering project and evaluate the sustainability of developed software, taking into consideration the societal and environmental aspects</i>		Total Marks	
Selection of Software Engineering Models: Process model selection and presents sufficient evidence to support argument for the model selection		[5 Marks]	
Role identification and Responsibility Allocation: Well-planned project with proper role identification and responsibility allocation in the project management activities		[5Marks]	
Formatting and Submission: Submission, Defense, Completeness, Spelling, grammar, and Organization of the Project report		[5Marks]	
CO4 (PO-k-1) <i>Apply engineering management principles and economic decision making to develop software engineering project management plan.</i>		Total Marks	
Project WBS and Testcases: Relevant WBS (project task list) and testcases for the proposed project are stated properly.		[5Marks]	
Effort Estimation and Scheduling: Project estimation was described using proper effort estimation or schedules based on available project resources		[5Marks]	
Risk Management: Sufficient and appropriate risks are identified, analyzed, and properly categorized or prioritized.		[5Marks]	

1. PROJECT PROPOSAL

1.1 Background to the Problem

Education outside of school and university has become an essential part of academic life in Bangladesh. Many university students earn income and gain experience by offering private tuition or running small coaching centers. While the demand for such services is increasing, the way they are managed is still very traditional and inefficient.

Most tutors rely on manual methods such as:

- Writing attendance in notebooks.
- Keeping payment records in ledgers or Excel files.
- Sharing class notes through scattered platforms like Messenger, WhatsApp, or Google Drive.

This fragmented system creates multiple problems:

- **Time wastage** – Tutors spend unnecessary time managing records instead of teaching.
- **Missed payments** – Without proper tracking, students may delay or avoid paying fees.
- **Poor organization** – Notes and materials get lost across different apps.
- **Limited scalability** – It is difficult for a tutor to manage more students or grow into a professional coaching setup.

Mentors Diary directly addresses these challenges. It provides a **one-stop platform** where tutors can manage students, record attendance, share content, collect fees through bank/Google Pay/bKash/Nagad, and communicate with both students and parents. By centralizing all tasks into a single dashboard, it helps tutors become more organized, professional, and scalable.

1.2 Solution to the Problem and Process Model Selection

1.1 Project Scope and Features

Mentors Diary covers the most critical tasks of a tutor's workflow:

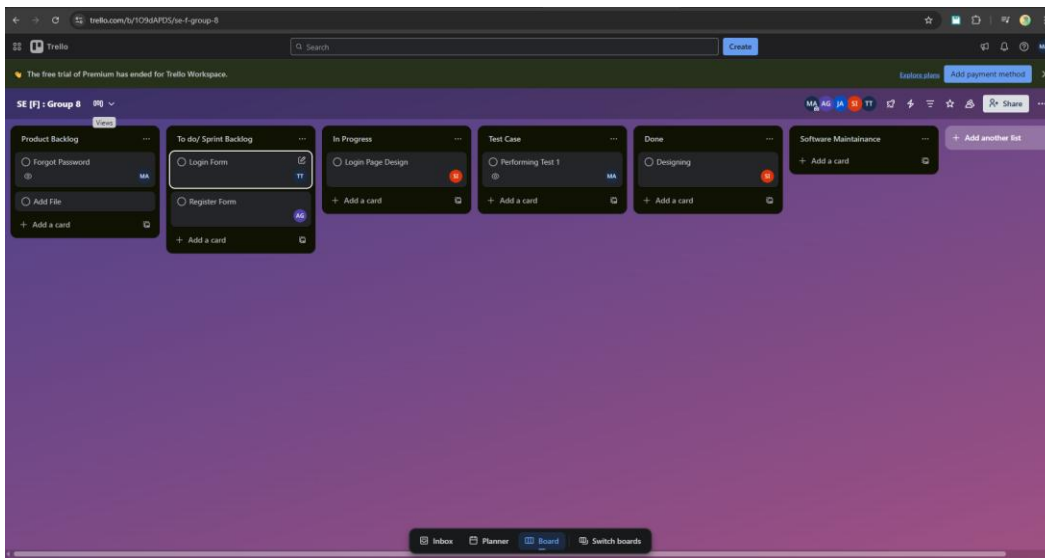
1. **Authentication & Role Management** – Secure login, registration, and password recovery for tutors, students, and parents.
2. **Dashboard** – Tutors can add, edit, remove, and view student profiles.
3. **Attendance Management** – Tutors can mark attendance and view history reports.
4. **Content Sharing** – Tutors can upload notes, assignments, and recordings. Students can view/download them.
5. **Fee Management** – Track individual and multiple student payments, record both online and offline transactions.

6. **Subscription Model** – Free plan (up to 3 students) for small tutors, and premium plan (up to 100 students) for larger setups.

1..2 User Story Table

Here’s the expanded explanation of user needs:

As a/an...	I want to...	So that...	Acceptance Criteria
Tutor	Add a new student	I can manage their profile, fees, and attendance	Student record saved, visible in dashboard
Tutor	Mark attendance	I can track student presence	Attendance saved with date/time; visible in reports
Tutor	Upload class material	Students can access learning resources easily	File uploaded, metadata stored, visible in content list
Student	Download content	I can review materials anytime	File download works with proper access
Parent	View fee summary	I can track my child’s payment status	Correct paid/due status shown in dashboard
Tutor	Collect payments via bKash/Nagad	I can receive fees securely	Payment recorded, transaction ID stored
Tutor	Upgrade subscription	I can manage more students	Subscription status updated, new limit applied



Existing Software Solutions

Some apps currently used by tutors include:

- **WhatsApp/Messenger/Google Drive** – Used for note sharing and communication, but no fee or attendance management.
- **Zoom/Google Meet** – Great for classes but no local payment or record-keeping support.
- **Moodle, Udvash, Retina** – Very advanced but expensive and complex, not suitable for individual tutors or small coaching centers.

None of these provide a **localized, integrated solution** for Bangladeshi tutors with features like bKash/Nagad payments and simple dashboards.

1.3 Selected Software Development Process Model

We selected **Extreme Programming (XP)** because:

- Tutors' requirements may **change frequently** as they test the system. XP supports this.
- Our **team size is small (3–5)**, and XP works best for small teams.
- XP focuses on **fast, incremental releases** so we can deliver a working MVP early.
- Quality is ensured through **pair programming and test-driven development**.

1.4 Project Environment & Requirements

- **Stable:** The main idea (attendance, notes, payments) is fixed.
- **Changing:** The details (fee types, discounts, reminders) will evolve with tutor feedback. XP adapts well to this balance of stable vision + changing details.

1.5 Team Support, Feasibility, and Communication

- XP practices like **daily standups, pair programming, and on-site customer** fit perfectly.
- Communication is continuous and direct with tutors.
- The solution is **feasible** because we can launch a Minimum Viable Product (MVP) quickly and improve it based on real usage.

1.6 Flexibility of XP

- **Scope changes:** Can be added in the next iteration.
- **Technology updates:** Refactoring keeps code clean and easy to upgrade.
- **User requirements:** Direct tutor involvement ensures changes are reflected fast.

1.7 Creative Solution to Real-Life Problem

1.8 Our creativity lies in:

- **Localization:** Supporting bKash and Nagad, familiar to Bangladeshi users.
- **Freemium model:** Tutors can start free (3 students) and later upgrade.
- **All-in-one dashboard:** No need to use multiple apps for notes, attendance, fees, and communication.

1.9 Target Users and Benefits

- **Tutors:** Save time, manage payments, look professional.
- **Students:** Access organized materials and updates.
- **Parents:** Transparent fee and attendance reports.

- **Coaching Admins:** Affordable way to manage operations without enterprise software.

1..10 Contribution to Scientific Results

Mentors Diary will generate **real, measurable data**:

- **Efficiency gains:** Hours saved per week in admin work.
- **Financial impact:** Reduction in unpaid fees due to reminders and digital tracking.
- **Academic research:** First localized EdTech study for small-scale tutors in Bangladesh.

1..11 Risk Management in XP

- **Wrong product risk** → Reduced by on-site tutor involvement.
- **Poor quality risk** → Controlled by TDD and pair programming.
- **Schedule risk** → Handled with short iterations and velocity tracking.

1..12 Justification of Model Selection

- **Waterfall** → Too rigid, needs fixed requirements.
- **Spiral** → Heavy documentation and overhead, not suitable for small teams.
- **XP** → Flexible, fast, high quality, and perfect for evolving tutor requirements.

• 1.3 Project Role Identification and Responsibilities

1..1 Main Roles

1. **Project Manager** – Oversees schedules, risk, and overall coordination.
2. **System Analyst** – Works with tutors/students to gather and refine requirements.
3. **UI/UX Designer** – Creates user-friendly interfaces for dashboards and content flows.
4. **Backend Developer** – Builds server APIs, manages databases, integrates payments.
5. **Frontend Developer** – Develops web and mobile apps using Flutter/React.
6. **QA/Test Engineer** – Designs test cases, runs TDD and ensures quality.
7. **DevOps Engineer** – Handles deployment, updates, and continuous integration (CI/CD).

1..2 Responsibilities at Key Stages

- **Requirements Gathering:** Analyst with tutors, supported by PM.
- **Design:** UI/UX Designer and Analyst create mockups; developers check feasibility.
- **Implementation:** Backend + Frontend developers write code, practice pair programming.
- **Testing:** QA + developers conduct unit, integration, and user acceptance testing.
- **Deployment:** DevOps ensures smooth hosting and updates.

1.3 Responsibility Distribution & Justification

- Analyst focuses on requirements because of strong communication skills.
- Developers share coding work for knowledge transfer and maintainability.
- QA ensures data security and reliability (important due to payment features).
- DevOps ensures availability and scalability.

This distribution ensures each stage is handled by those most skilled, while collaboration keeps the project efficient.

2. Functional Requirements

1. Description of Major Functionalities

Mentors Diary is an all-in-one platform for university student tutors and coaching centers that simplifies daily operations. The system's major functionalities are designed to streamline key administrative tasks, allowing tutors to focus on teaching. The platform includes a Student Management System to handle student data and academic progress, a Digital Content Sharing module for notes and resources, an Integrated Payment System for fee collection, a Communication Dashboard for interacting with students, and an Administrative Overview Panel for monitoring operations. The project's primary objective is to create an intuitive and efficient digital platform that addresses challenges like disorganized student data, inefficient material distribution, and difficulties with fee collection.

2. Core, Service Operations and Features How it Supports Project Objectives

Authentication

Login, Register, Forgot Password

Ensures a secure environment for tutors, students, and parents to access the platform and their respective data.

Student Management

Add/Edit/Remove Student , View All Students , View Student Profile

Helps tutors efficiently manage student information and records in a single location, reducing manual workload and disorganization.

Attendance Tracking

Take Attendance , List of attendance , Automated tracking

Reduces administrative burden and manual errors, ensuring tutors have a reliable record to prevent issues like unverified attendance and missed payments.

Content Sharing

Create/Upload/Update Content (Notes, Assignments, Recordings) , View/Download Content

Provides a centralized and organized way to distribute study materials, replacing disorganized methods like sharing through messaging apps.

Fee Management

Track fee payment, view total paid/due/remaining balance , add/remove payment statements , integrated fee collection

Simplifies tuition fee collection through popular local and global payment channels, minimizing the risk of fraud or mishandling of funds and directly increasing a tutor's monthly income.

Subscription Management

Free Limited Subscription (max 3 students) , Premium Subscription (max 100 students)

Supports a sustainable business model by providing a low-cost subscription that allows tutors to scale their services while ensuring they keep what they earn.

3. How Functions Support Project Objectives

Each function in Mentors Diary is designed to solve a specific problem identified in the tutoring ecosystem, making the service more efficient, professional, and scalable.

Student Management System: By allowing tutors to manage student data, academic progress, and attendance in one place, this function directly addresses the issue of disorganized student data. It reduces the manual workload and the need for multiple apps, which frees up a tutor's time to focus on teaching.

Digital Content Sharing: This module provides a secure and organized way to distribute class notes, assignments, and other learning materials. It tackles the challenge of inefficient material distribution that occurs when tutors use informal and disorganized platforms like social media.

Integrated Payment System: This feature simplifies fee collection through multiple channels like bKash, Nagad, and Google Pay. It is a direct solution to the difficulties in fee collection , preventing money loss from forgotten or missed payments. It also helps tutors track payments and directly increases their monthly income.

Communication Dashboard: This functionality ensures clear and consistent interaction between tutors, students, and parents. This helps overcome the problem of limited communication channels and the disorganization that comes from using social media platforms for communication.

Administrative Overview Panel: This gives tutors and small coaching centers oversight and control of daily operations. It helps them move from manual, inefficient processes to a more organized and professional setup, which is essential for scaling their tutoring service.

4. Workflow Descriptions

Workflow for a Tutor Adding a Student: A tutor logs in and navigates to the **Dashboard**. From there, they click on "Add student". They are then directed to a registration page where they enter the student's name, parent's name, institution, and contact information. After filling out the required fields, the tutor clicks "Save" to complete the process. This new student is then visible in the "View All Students" list.

5. Acceptance Criteria

Acceptance criteria are the conditions that must be met for each functional requirement to be considered complete and working as intended.

User Registration: A user (tutor, student, or parent) must be able to successfully create a new account by filling out all required fields. The system must validate the information provided and confirm the user's registration.

Fee Collection: When a tutor records a payment for a student, the system must automatically update the student's "Total Paid," "Total Due," and "Remaining Balance". The payment must be saved in the student's payment history with the correct date and time. A financial report must be available to prove how the app directly increases a tutor's monthly income.

Content Upload: A tutor must be able to upload a file (e.g., a PDF of notes or a video lecture) to a class. The uploaded content must be immediately accessible to all students in that class for viewing and downloading, and the content must be categorized correctly.

Attendance Tracking: A tutor must be able to take attendance for a class, and the system should automatically log and save the attendance data. The attendance record must be viewable in a list format, and it should reduce the manual administrative burden on the tutor

2.2 Non-Functional Requirements

1. Quality Attributes

Mentors Diary will prioritize

usability, reliability, security, and scalability to ensure it provides a professional, dependable, and growing service.

2. Performance

The system should provide a responsive user experience. Pages like the student dashboard and content lists should load within 2-3 seconds. Payment processing and attendance logging should be near-instantaneous to avoid frustration for users.

3. Reliability

The platform should be designed for high availability, with minimal downtime. It will use a reliable backend infrastructure, such as Firebase, that offers stability and data synchronization. Secure login systems and encrypted storage will ensure data integrity and privacy, minimizing the risk of data loss.

4. Integrity/Security

Mentors Diary will use encrypted storage and secure login systems to protect all user data, including academic records, payment history, and personal information. Access to student data will be restricted to authorized personnel only, such as the tutor and the student themselves. The integrated payment system will rely on secure, industry-standard protocols to minimize the risk of fraud.

5. Usability

The user interface must be intuitive and straightforward, catering to users who may not be tech-savvy. The design prioritizes ease of use to simplify processes that were traditionally manual. The platform will also include support for local languages and culturally familiar services, making it more accessible.

6. Maintainability

The system will be built using modern, open-source frameworks like which are easily accessible and maintainable. This approach allows for future modifications, bug fixes, and upgrades with minimal friction. The modular design of the core functionalities will also make it easier to add new features or adjust existing ones without affecting the entire system.

7. Scalability

Mentors Diary is designed to be scalable to support an increasing number of users, data, and features. The subscription model, which limits students per tutor, allows the system to manage growth incrementally. The backend infrastructure will be able to handle a large number of student profiles, attendance records, and payment transactions as the user base expands.

3. PROJECT ESTIMATION AND SCHEDULING

1. Project Scope

Our project **Mentors Diary** is a mobile + web platform that helps tutors manage students, take attendance, share notes, and collect fees.

The main modules are:

- **Authentication** (Login, Register, Forgot Password)
- **Tutor Dashboard** (Add/Edit/Remove Students, View Profiles)
- **Attendance Management**
- **Content Sharing** (Notes, Assignments, Recordings)
- **Fee Management** (Track Fees, Payment History, Integrated Payments)
- **Subscription System** (Free Plan, Premium Plan)

2) LOC Method

Given:

- $LOC = 100,000$
 - $Productivity = 500 \text{ Lperson-month}$
- Effort (PM) = LOC / Productivity**

$$Effort = 100,000 / 500 = 200 \text{ person-months}$$

$$Effort = 100,000 / 500 = 200 \text{ person-months}$$

Person-hours:

$$200 \times 160 = 32,000 \text{ hours}$$

3) COCOMO Semi-Detached Estimation

Constants:

- $Coefficient = 3.0$
- $p = 1.12$
- $C = 2.5$
- $T = 0.35$

KLOC = 100

Step 1: Effort (person-months)

$$Effort = Coefficient \times (KLOC)^p$$

$$Effort = 3.0 \times (100)^{1.12}$$

$$Effort, PM = 3.0 \times 165.0 \approx 495$$

Person-hours:

$$495 \times 160 = 79,200 \text{ hours}$$

Step 2: Development Time (months)

$$DM = C \times (Effort)^T$$

$$DM = 2.5 \times (495)^{0.35}$$

$$DM = 2.5 \times 6.96 \approx 17.4 \text{ months} \approx 1.45 \text{ years}$$

Step 3: Require no of people

$$ST = \text{Effort} / DM = 495 / 17.4 \approx 29$$

people

Sure! Here's a simpler, student-friendly version of assumptions and variations:

Assumptions and Possible Changes

1. Lines of Code (LOC)

We think the project will have 100,000 lines of code.

Real code might be more or less, depending on how we write it.

2. Productivity

We assume one person can do 500 lines of code per month.

If team members work faster or slower, the time will change.

3. COCOMO Model

We used Semi-Detached mode for medium complexity.

It assumes work includes testing, debugging, and integration.

Student teams might take more time because we are learning and coordinating.

4. Team and Time

Calculations assume everyone works efficiently.

If some members are busy or slower, the project may take longer.

5. Other Factors

Adding new features or facing technical problems can increase the time and effort.

6. General

These are best guesses, not exact numbers.

They help plan work, but real results may differ.

Summary for Report

We assumed 100,000 lines of code, average speed of 500 LOC per month, medium complexity, and efficient teamwork. Actual effort, time, or team size may change if features are added or problems happen.

Project Scheduling for Mentors Diary

1) Breaking the Project into Tasks and Assigning Responsibilities

We divide the project into smaller tasks, connect them, and assign team members:

Task	Description	Responsible
Requirement Analysis	Gather all project requirements	Team Lead
System Design	Design database, screens, and flow	Designer & Developer
Coding Frontend	Implement UI for web & mobile	Frontend Developer
Coding Backend	Implement server, database, API	Backend Developer
Attendance Module	Coding and integration	Developer
Content Sharing Module	Coding and integration	Developer
Fee Management Module	Coding & payment integration	Developer
Subscription System	Implement free & premium plans	Developer
Testing & Debugging	Unit & integration testing	All Members
Deployment	Launch web & mobile app	All Members

2) Effort Allocation Across Phases

We use the 40–20–40 guideline:

- **Analysis & Design:** 40% of total effort
- **Coding/Implementation:** 20% of total effort
- **Testing & Deployment:** 40% of total effort

Example: If total effort = 200 person-months (from LOC estimate):

- Analysis & Design = $200 \times 0.4 = 80$ person-months
- Coding = $200 \times 0.2 = 40$ person-months
- Testing & Deployment = $200 \times 0.4 = 80$ person-months

3) Schedule and Milestones

- **Milestone 1:** Requirements and Design complete → Month 2
- **Milestone 2:** Core Modules (Dashboard, Attendance, Content) → Month 5
- **Milestone 3:** Fee & Subscription Modules → Month 7
- **Milestone 4:** Testing & Deployment → Month 9

4. Project Milestones and Major Deliverables

Milestone	Time (Month)	Major Deliverables / Outputs	Responsible
Milestone 1: Requirements & Design	Month 1–2	- Complete list of requirements - System design documents - Database schema - Wireframes & UI mockups	Team Lead + Designer
Milestone 2: Core Modules Development	Month 3–5	- Authentication module (Login/Register/Forgot Password) - Tutor Dashboard (Add/Edit/View Students) - Attendance Module (Take & View Attendance) - Content Sharing Module (Upload/Download Notes & Assignments) - Fee Management Module (Track & Integrate Payments)	Frontend + Backend Developers
Milestone 3: Advanced Modules Development	Month 5–7	- Subscription System (Free & Premium Plan) - Unit & Integration Testing Reports	Backend + Full Stack Developers
Milestone 4: Testing & Deployment	Month 7–9	- Bug Fixing & Optimization - Live Deployment on Web & Mobile - User Manual & Documentation - Final Project Presentation	All Team Members
Milestone 5: Final Review & Delivery	Month 9	- Complete Project Report - Screenshots & Demo Videos - Feedback Collection & Final Adjustments	Team Lead + All Members

5) **Earned Value Analysis (EVA)**
Assume:

- BAC = 200
- BCWS (planned for this month) = 50
- BCWP (actual completed) = 40

- ACWP (actual cost spent) = 45

Planned Value (BCWS) & Percent Scheduled:

Percent scheduled = $BCWS / BAC$

Percent scheduled = $50 / 200 = 25\%$

Shows how much work is actually done.

Percent

Complete:

Percent complete = $BCWP / BAC$

Percent Complete = $40 / 200 = 20\%$

Shows how much work is actually done.

Schedule Performance Index (SPI):

$SPI = BCWP / BCWS$

$SPI = 40 / 50 = 0.8$, Slightly behind schedule

- $SPI > 1 \rightarrow$ Ahead of schedule
- $SPI < 1 \rightarrow$ Behind schedule

Schedule Variance (SV):

$SV = BCWP - BCWS$

$SV = 40 - 50 = -10 \rightarrow$ Delay of 10 person-months

- Positive SV $\rightarrow$ ahead, Negative SV $\rightarrow$ delay

Cost Performance Index (CPI):

$CPI = BCWP / ACWP$

$CPI > 1 \rightarrow$ cost-efficient

$CPI < 1 \rightarrow$ over budget

$CPI = 40 / 45 = 0.89 \rightarrow$ Cost efficiency is below 1, over budget

Cost Variance (CV):

$CV = BCWP - ACWP$

$CV = 40 - 45 = -5 \text{ PM} \rightarrow$ Over budget by 5 person-months

Positive CV $\rightarrow$ under budget, Negative CV $\rightarrow$ over budget

Common Causes of Delay

- Unrealistic deadlines
- Changing project requirements

- Technical difficulties
- Team or human errors
- Miscommunication

Planning ahead and regular progress meetings can reduce these delays.

4. SOFTWARE DESIGN

Use-Case Diagram



System Name:

Mentors Diary System

• Actors:

1. Tutor
 - o Responsible for managing student-related and academic operations.
2. Student
 - o Uses the system to access learning content, communicate, and track progress.
3. Parent/Guardian
 - o Monitors student progress and manages fee payments.
4. Administrator
 - o Ensures overall system management, mainly through generating reports.

- **Use Cases and Actor Interactions:**

Tutor Functions:

- Manage Student Profiles: Add, update, or modify student details.
- Track Attendance: Record and monitor student attendance.
- Manage Digital Content: Upload, organize, and share study materials or resources.
- Manage Fees: Oversee fee structures and payments.
- Communicate with Users: Exchange messages or updates with students and parents.

Student Functions:

- Communicate with Users: Interact with tutors or peers.
- Access Shared Content: View digital learning resources uploaded by tutors.
- View Academic Progress: Track their performance and progress.

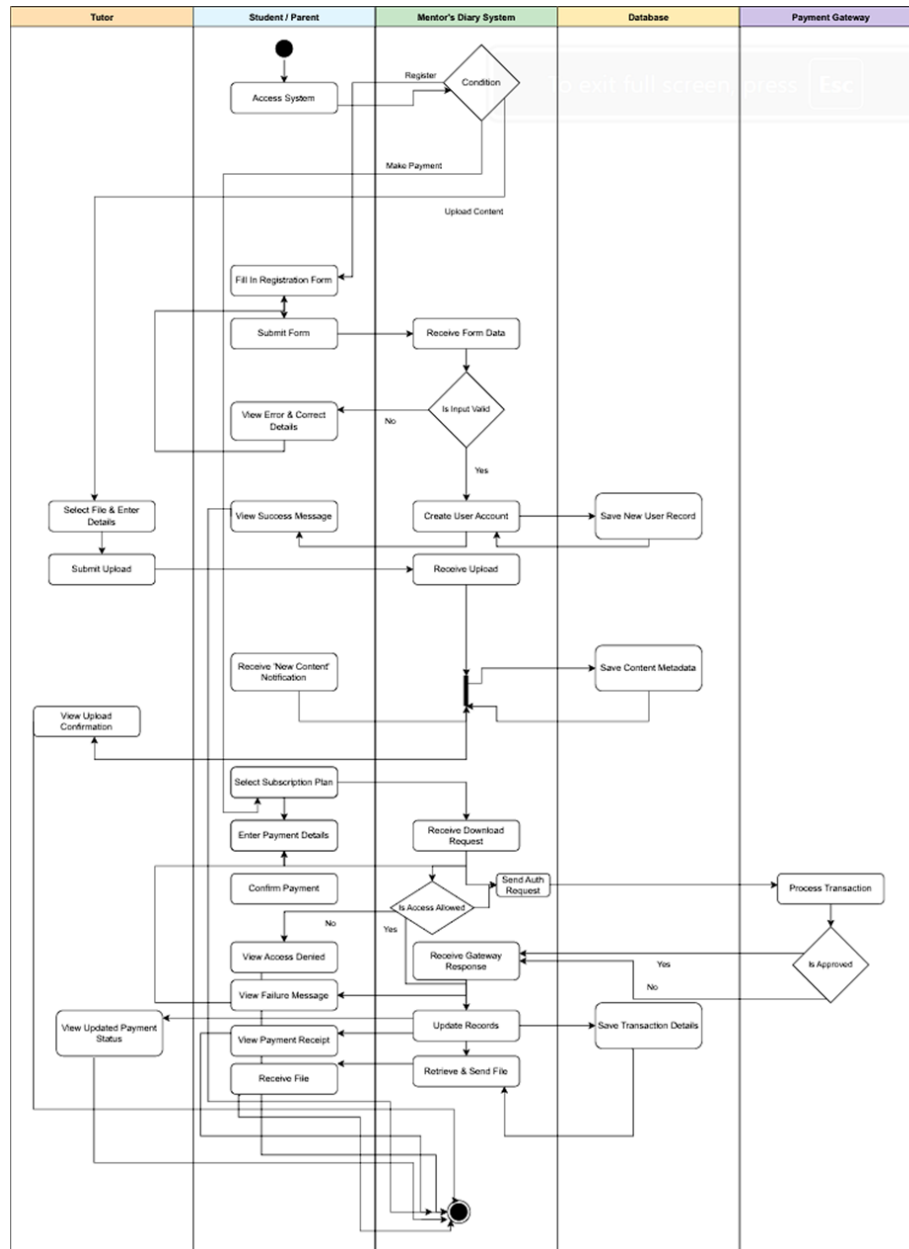
Parent/Guardian Functions:

- Communicate with Users: Connect with tutors for student-related queries.
- View Academic Progress: Monitor student's learning outcomes.
- Make Fee Payment: Pay student fees directly through the system.

Administrator Functions:

- Generate Reports: Create administrative or academic reports for analysis.

Activity Diagram



1. User Registration

- A Student/Parent accesses the system.
- They fill in the registration form and submit it.
- System checks if the input is valid:
 - o If invalid → user sees an error message, corrects, and resubmits.
 - o If valid → system creates user account and saves the new user record in the database.
- Users receive a successful message.

2. Content Upload (Tutor)

- A Tutor selects a file and enters details.
- Submits the upload request.
- System saves content metadata in the database.
- Tutor receives an upload confirmation.
- Students/Parents later receive “new content available” notifications.

3. Payment Process

- A Student/Parent selects a subscription plan.
- They enter payment details and confirm payment.
- System sends details to the Payment Gateway.
- Gateway processes transactions:
 - If approved → system saves transaction details in the database, updates payment records, and user gets a payment receipt.
 - If not approved → system shows failure message.

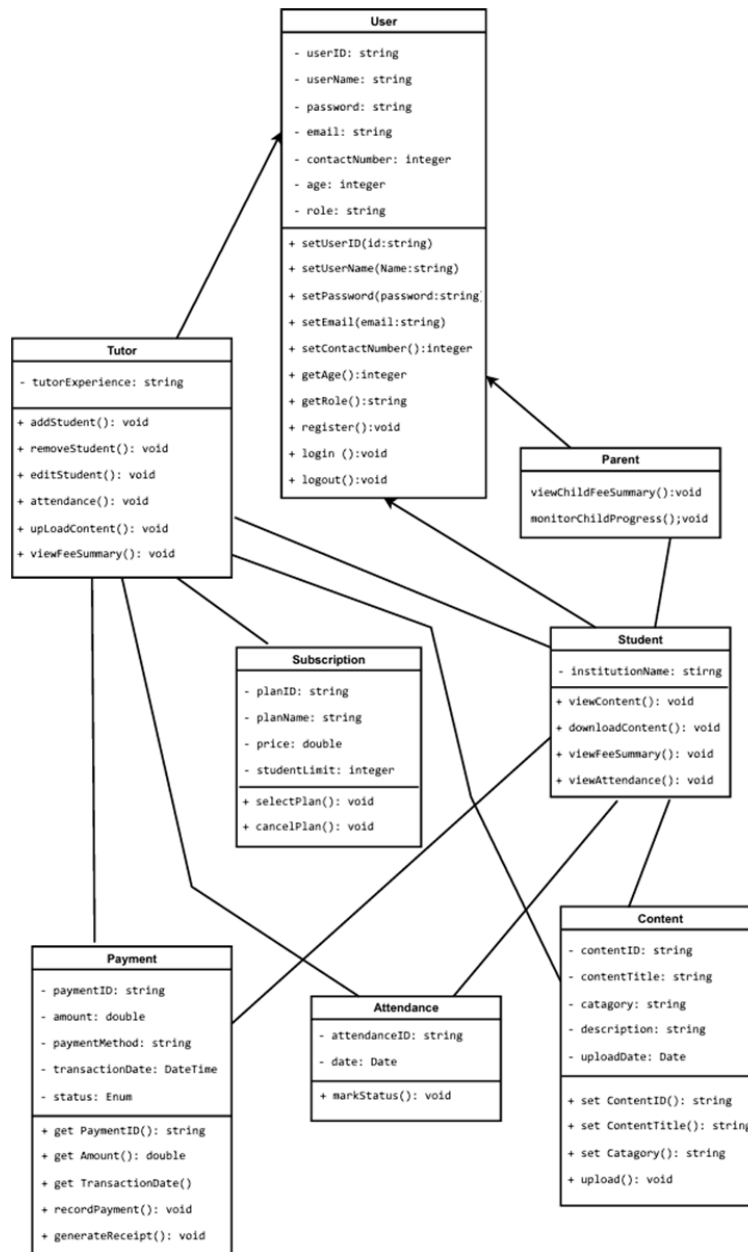
4. Content Access

- A Student/Parent can search or browse for content.
- When clicking “Download”, system checks if access is allowed (based on subscription/payment).
 - If allowed → file is retrieved from the database and user receives the file.
 - If denied → system shows an “Access Denied” message.

5. Actors

- Tutor → Uploads content.
- Student/Parent → Registers, pays, and downloads content.
- Database → Stores user records, content metadata, and transactions.
- Payment Gateway → Handles payment approval.

Class Diagram



1. User (Base Class)

- Attributes:
 - o userID: string
 - o userName: string
 - o password: string
 - o email: string
 - o contactNumber: integer
 - o age: integer

- o role: string
- Methods:
 - o setUserID(id: string)
 - o setUsername(name: string)
 - o setPassword(password: string)
 - o setEmail(email: string)
 - o setContactNumber(): integer
 - o getAge(): integer
 - o getRole(): string
 - o register(): void
 - o login(): void
 - o logout(): void

2. Parent (Inherits User)

- Methods:
 - o viewChildFeeSummary(): void
 - o monitorChildProgress(): void

3. Student (Inherits User)

- Attributes:
 - o institutionName: string
- Methods:
 - o viewContent(): void
 - o downloadContent(): void
 - o viewFeeSummary(): void
 - o viewAttendance(): void

4. Tutor (Inherits User)

- Attributes:
 - o tutorExperience: string
- Methods:
 - o addStudent(): void
 - o removeStudent(): void
 - o editStudent(): void
 - o attendance(): void
 - o uploadContent(): void
 - o viewFeeSummary(): void

5. Subscription

- Attributes:
 - o planID: string
 - o planName: string
 - o price: double
 - o studentLimit: integer
- Methods:
 - o selectPlan(): void
 - o cancelPlan(): void

6. Payment

- Attributes:
 - o paymentID: string
 - o amount: double
 - o paymentMethod: string
 - o transactionDate: DateTime
 - o status: Enum
- Methods:
 - o getPaymentID(): string
 - o getAmount(): double
 - o getTransactionDate()
 - o recordPayment(): void
 - o generateReceipt(): void

7. Attendance

- Attributes:
 - o attendanceID: string
 - o date: Date
- Methods:
 - o markStatus(): void

8. Content

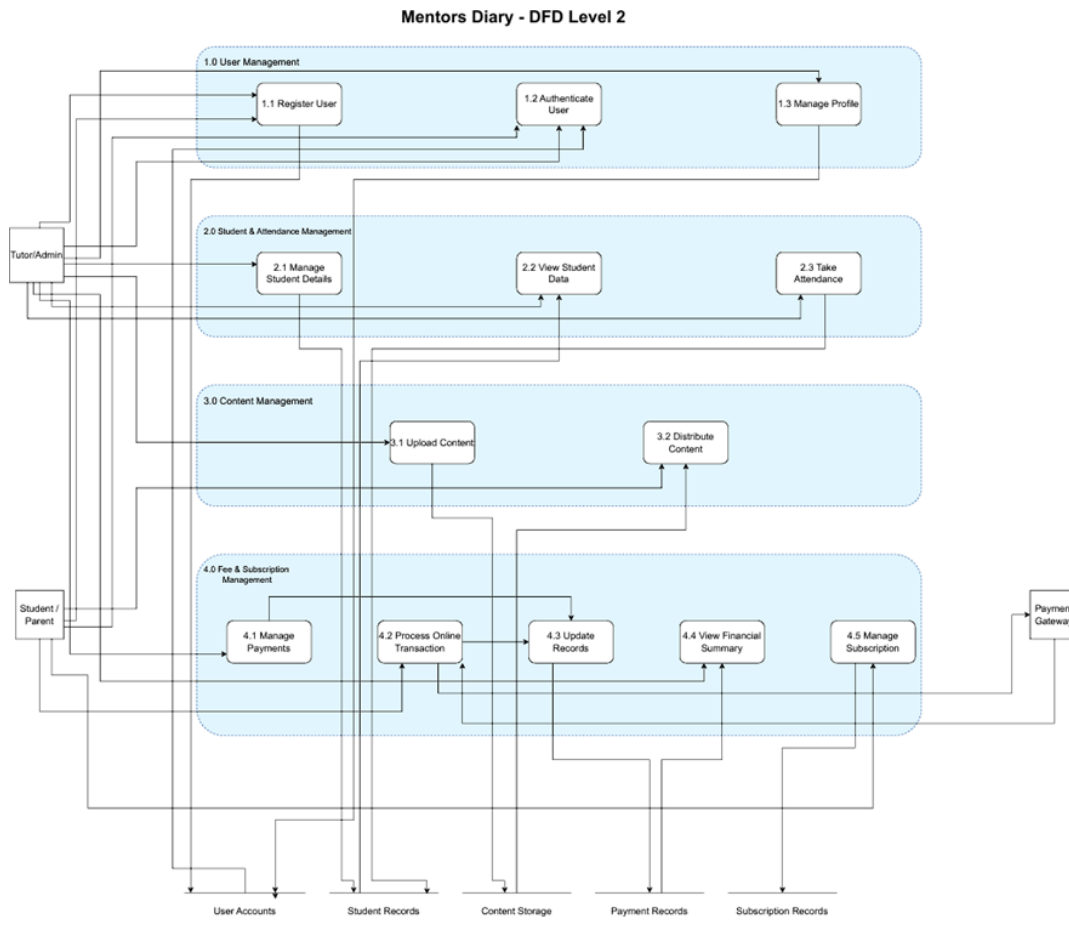
- Attributes:
 - o contentID: string
 - o contentTitle: string
 - o category: string
 - o description: string
 - o uploadDate: Date
- Methods:
 - o setContentID(): string

- o setContentTitle(): string
- o setCategory(): string
- o upload(): void

9. Relationships (Implied in Diagram)

- Inheritance:
 - o User → Parent, Student, Tutor.
- Associations:
 - o Tutors manage Students.
 - o Students/Parents are linked to Subscription and Payment.
 - o Tutors upload Content.
 - o Tutors mark Attendance for Students.

Data Flow Diagram (DFD)



Actors

- **Tutor/Admin**
- **Student / Parent**
- **Payment Gateway**

• System Modules and Processes

1.0 User Management

- 1.1 Register User – Handles new user registration.
- 1.2 Authenticate User – Manages login and authentication.
- 1.3 Manage Profile – Allows updating personal details.

2.0 Student & Attendance Management

- 2.1 Manage Student Details – Stores and updates student information.
- 2.2 View Student Data – Displays student-related records.
- 2.3 Take Attendance – Records and tracks attendance.

3.0 Content Management

- 3.1 Upload Content – Tutors/Admin upload study materials.
- 3.2 Distribute Content – Provides content access to students/parents.

4.0 Fee & Subscription Management

- 4.1 Manage Payments – Oversees payment records.
- 4.2 Process Online Transaction – Connects with the payment gateway for transactions.
- 4.3 Update Records – Updates payment/subscription status.
- 4.4 View Financial Summary – Displays reports on financial activity.
- 4.5 Manage Subscription – Handles subscription plans and renewals.

5.0 Data Stores

- User Accounts – Stores login and profile information.
- Student Records – Keeps student details and attendance.
- Content Storage – Repository of uploaded study materials.
- Payment Records – Stores transaction details.
- Subscription Records – Stores subscription status/history.



Welcome to Mentors Diary : Modern Education, Made Simple

Register as a Student

Student's Name:

Father's Name:

Mother's Name:

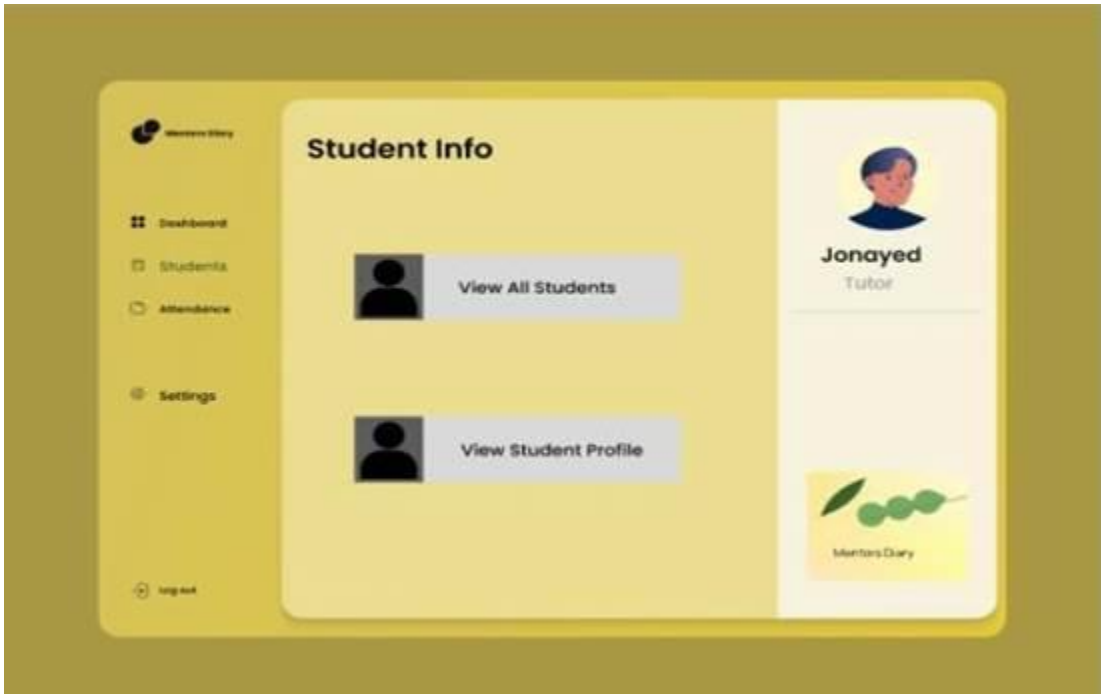
Institution Name:

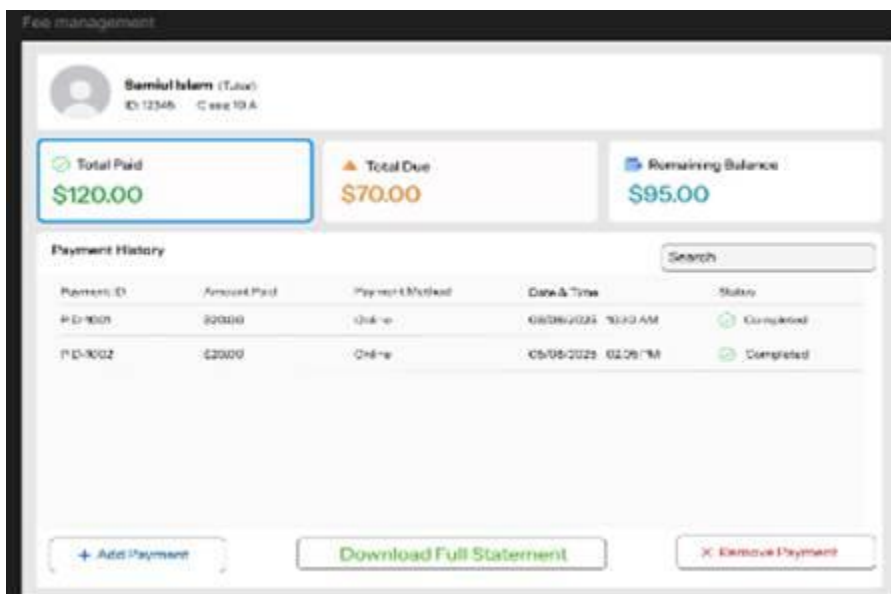
Gender: ☐ Male ☐ Female Age:

Contact No:

Email:

Gardian Contact No:





Student fee details

Samiul Islam

ID: 12345 Class: 10 A

Total Paid

\$120.00

Total Due

\$70.00

Next Due Date

Oct 08, 2025

Payment History

Payment ID	Amount Paid	Payment Method	Date & Time	Status
PD-1001	\$20.00	Online	03/08/2025 10:30 A.M	Completed
PD-1002	\$20.00	Online	05/08/2025 02:05 P.M	Completed

Make Payment

Download Receipt

Subscription - 1

Select a Plan

₹200 per Month

Basic

Ad-Free

Unlimited Streaming

One user only

Watch up to 720p Resolution

Select Plan

₹500 per Month

Standard

Ad-Free

Unlimited Streaming

Maximum of 3 Students

Watch up to 1080p Resolution

Select Plan

₹1000 per Month

Premium

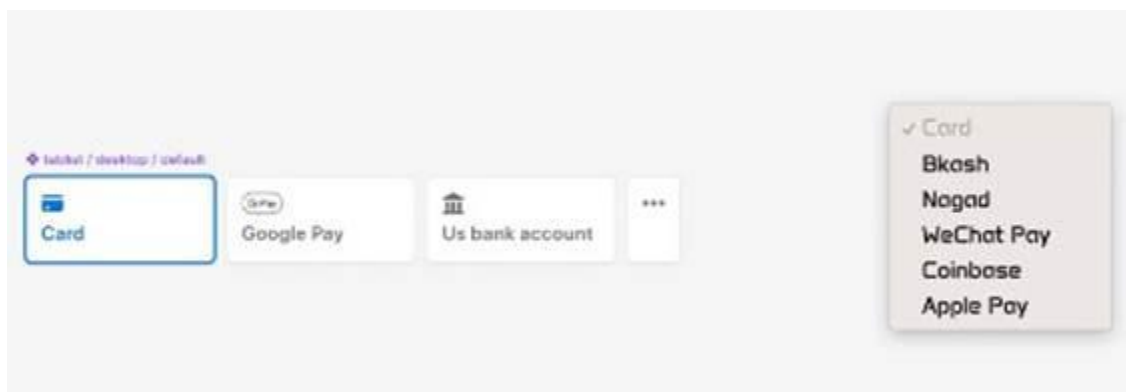
Ad-Free

Unlimited Streaming

Maximum of 100 Students

Watch up to 4k Resolution

Select Plan



Payment Receipt

Success

Mentors Diary

Dhaka, Bangladesh

Transaction ID:

TXN-2024-001234567

Date & Time:

8/26/2025, 8:30:00 PM

Payment Method

Visa Credit Card **** 4532

Items

Basic	BDT 200
-------	---------

Subtotal

BDT 200

Total

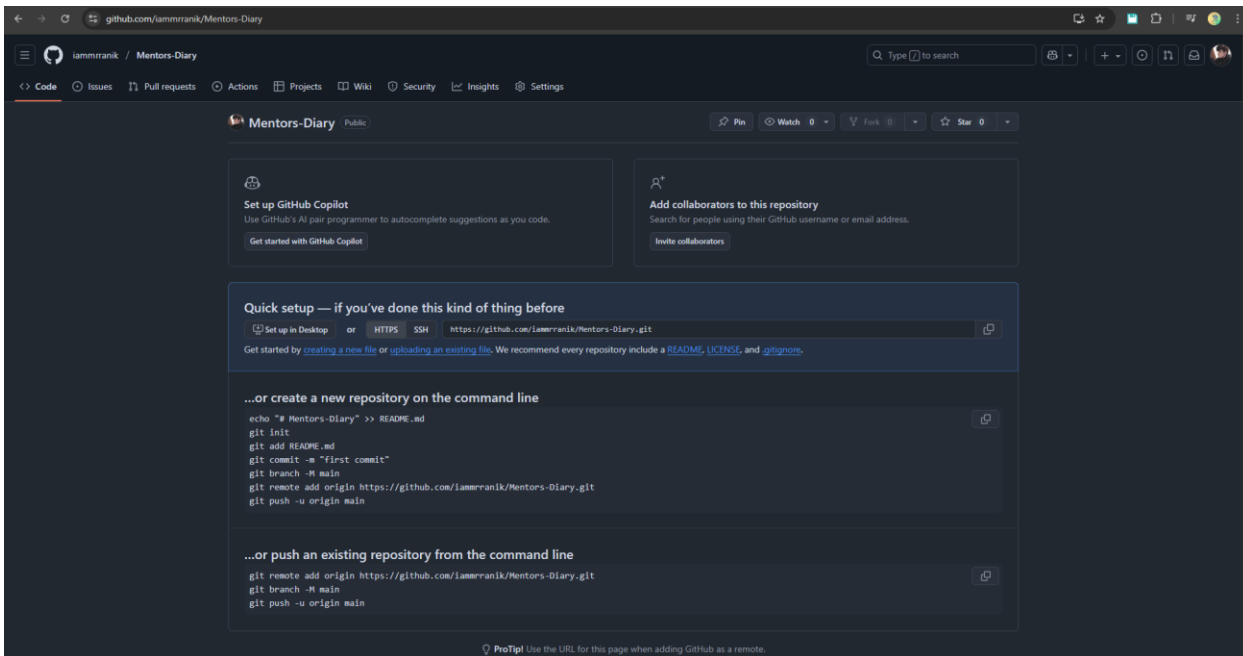
BDT 200

Thank you for your payment!

Keep this receipt for your records.

5. GIT WORKFLOW

a. Main Branch



```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19045.6332]
(c) Microsoft Corporation. All rights reserved.

C:\Projects\Mentors-Diary>echo "# Mentors-Diary" >> README.md

C:\Projects\Mentors-Diary>git init
Initialized empty Git repository in C:/Projects/Mentors-Diary/.git/

C:\Projects\Mentors-Diary>git add README.md

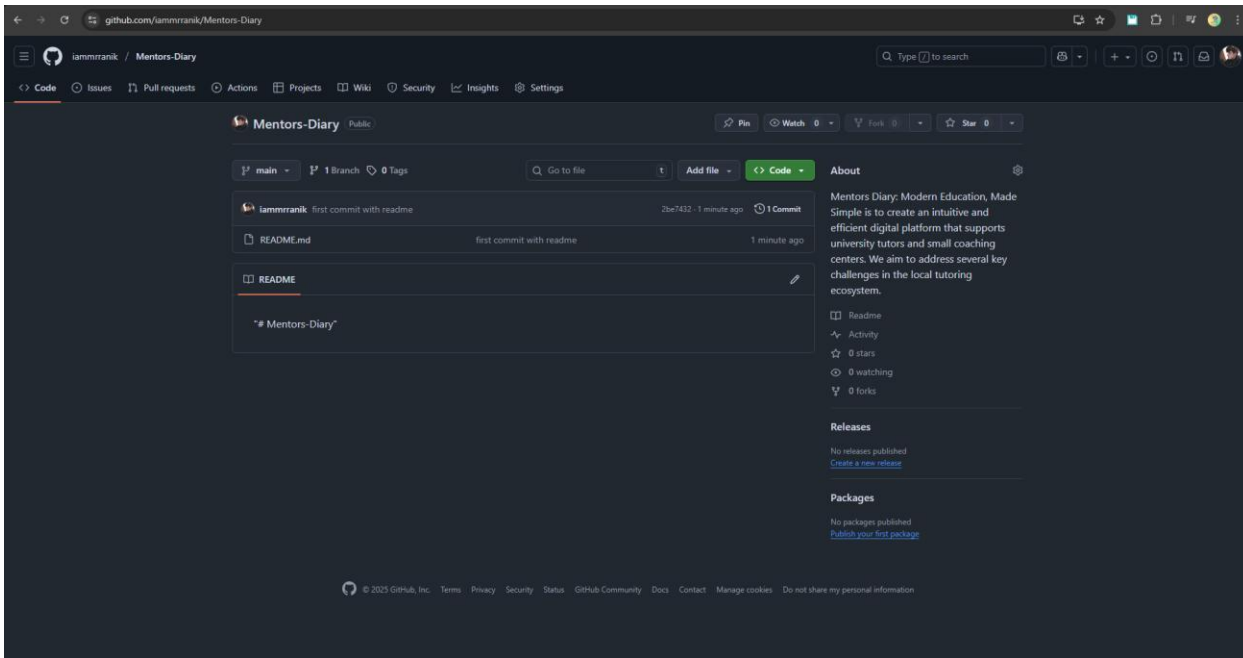
C:\Projects\Mentors-Diary>git commit -m "first commit with readme"
[master (root-commit) 2be7432] first commit with readme
 1 file changed, 1 insertion(+)
 create mode 100644 README.md

C:\Projects\Mentors-Diary>git branch -M main

C:\Projects\Mentors-Diary>git remote add origin https://github.com/iammrarik/Mentors-Diary.git

C:\Projects\Mentors-Diary>git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 235 bytes | 78.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/iammrarik/Mentors-Diary.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

C:\Projects\Mentors-Diary>
```



b. Branching (Jonayed)

```
C:\Windows\System32\cmd.exe
E:\>git clone https://github.com/iammrarik/Mentors-Diary
Cloning into 'Mentors-Diary'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

E:\>cd Mentors-Diary

E:\Mentors-Diary>git checkout -b feature/student-dashboard
Switched to a new branch 'feature/student-dashboard'

E:\Mentors-Diary>git add .

E:\Mentors-Diary>git commit -m "Design the initial student dashboard UI"
Author identity unknown

*** Please tell me who you are.

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (got 'HP@DESKTOP-I4QUGHQ.(none)')

E:\Mentors-Diary>git config --global user.email "jonayed592@gmail.com"

E:\Mentors-Diary>git config --global user.name "jonayed23-ui"

E:\Mentors-Diary>git commit -m "Design the initial student dashboard UI"
On branch feature/student-dashboard
nothing to commit, working tree clean

E:\Mentors-Diary>git commit -m "Design the initial student dashboard UI"
On branch feature/student-dashboard
nothing to commit, working tree clean

E:\Mentors-Diary>git push origin feature/student-dashboard
info: please complete authentication in your browser...
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
```

```
C:\Windows\System32\cmd.exe
E:\>git clone https://github.com/iammrranik/Mentors-Diary
Cloning into 'Mentors-Diary'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

E:\>cd Mentors-Diary

E:\Mentors-Diary>git checkout -b feature/student-dashboard
Switched to a new branch 'feature/student-dashboard'

E:\Mentors-Diary>git add .

E:\Mentors-Diary>git commit -m "Design the initial student dashboard UI"
Author identity unknown

*** Please tell me who you are.

Run

    git config --global user.email "you@example.com"
    git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (got 'HP@DESKTOP-I4QUGHQ.(none)')

E:\Mentors-Diary>git config --global user.email "jonayed592@gmail.com"

E:\Mentors-Diary>git config --global user.name "jonayed23-ui"

E:\Mentors-Diary>git commit -m "Design the initial student dashboard UI"
On branch feature/student-dashboard
nothing to commit, working tree clean

E:\Mentors-Diary>git commit -m "Design the initial student dashboard UI"
On branch feature/student-dashboard
nothing to commit, working tree clean

E:\Mentors-Diary>git push origin feature/student-dashboard
info: please complete authentication in your browser...
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
```

c. Branching (Tahomina)

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.26100.6584]
(c) Microsoft Corporation. All rights reserved.

D:\PROJECT>git clone https://github.com/iammrranik/Mentors-Diary.git
Cloning into 'Mentors-Diary'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

D:\PROJECT>cd Mentors-Diary

D:\PROJECT\Mentors-Diary>git checkout -b feature/content-table
Switched to a new branch 'feature/content-table'

D:\PROJECT\Mentors-Diary>git add .

D:\PROJECT\Mentors-Diary>git commit -m "Design the content-table feature"
On branch feature/content-table
nothing to commit, working tree clean

D:\PROJECT\Mentors-Diary>git push origin feature/content-table
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/content-table' on GitHub by visit
ing:
remote:      https://github.com/iammrranik/Mentors-Diary/pull/new/feature/co
ntent-table
remote:
To https://github.com/iammrranik/Mentors-Diary.git
 * [new branch]      feature/content-table -> feature/content-table

D:\PROJECT\Mentors-Diary>
```

d. Branching (Samiul)

```
C:\Windows\System32\cmd.exe
(c) Microsoft Corporation. All rights reserved.

D:\Summer 25\Software\GitLabTask>git clone https://github.com/iammrranik/Mentors-Diary.git
Cloning into 'Mentors-Diary'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

D:\Summer 25\Software\GitLabTask>cd Mentors-Diary

D:\Summer 25\Software\GitLabTask\Mentors-Diary>git checkout -b feature/payment-gateway
Switched to a new branch 'feature/payment-gateway'

D:\Summer 25\Software\GitLabTask\Mentors-Diary>git add .

D:\Summer 25\Software\GitLabTask\Mentors-Diary>git commit -m "Implemented the payment"
On branch feature/payment-gateway
nothing to commit, working tree clean

D:\Summer 25\Software\GitLabTask\Mentors-Diary>git push origin feature/payment-gateway
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/payment-gateway' on GitHub by visiting:
remote:   https://github.com/iammrranik/Mentors-Diary/pull/new/feature/payment-gateway
remote:
To https://github.com/iammrranik/Mentors-Diary.git
 * [new branch]      feature/payment-gateway -> feature/payment-gateway

D:\Summer 25\Software\GitLabTask\Mentors-Diary>
```

e. Branching (Argha)

```
C:\Windows\System32\cmd.exe
(c) Microsoft Corporation. All rights reserved.

D:\AIUB\GitLab Task>git clone https://github.com/iammrranik/Mentors-Diary.git
Cloning into 'Mentors-Diary'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

D:\AIUB\GitLab Task>cd Mentors-Diary

D:\AIUB\GitLab Task\Mentors-Diary>git checkout -b feature/login-interface
Switched to a new branch 'feature/login-interface'

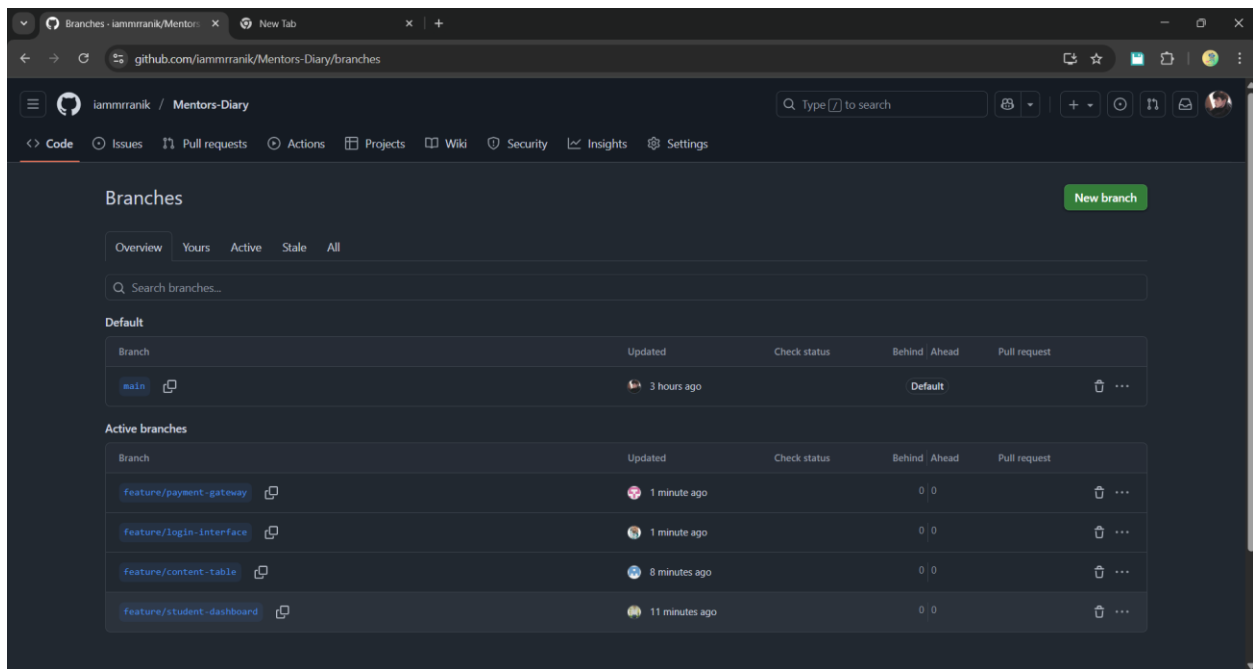
D:\AIUB\GitLab Task\Mentors-Diary>git add .

D:\AIUB\GitLab Task\Mentors-Diary>git commit -m "Designed the Login form and Registration form"
On branch feature/login-interface
nothing to commit, working tree clean

D:\AIUB\GitLab Task\Mentors-Diary>git push origin feature/login-interface
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/login-interface' on GitHub by visiting:
remote:   https://github.com/iammrranik/Mentors-Diary/pull/new/feature/login-interface
remote:
To https://github.com/iammrranik/Mentors-Diary.git
 * [new branch]      feature/login-interface -> feature/login-interface

D:\AIUB\GitLab Task\Mentors-Diary>
```

f. Total Branches



6. TESTING

Test ID	Test Case Objective	Pre requisite	Steps	Input Data	Expected Output	Actual Output	Status
TC_01	verify Uploads of Assignment	User must be logged in as Instructor	1.click "uploads Content" 2. Enter Title: "Assignment 1" 3. Select Category: Assignment 4. Click Upload	Title: Assignment 1, Category: Assignment	Assignment appears under "Assignments" tab	Assignment appears under "Assignments" tab	PASS
TC_02	Validate Search Functionality	Content must exist in system	1. Enter "History 101" in Search Bar 2. Press Enter	Search Query: History 101	"Class Materials: History 101" is displayed	"Class Materials: History 101" is displayed	PASS
TC_03	Test Upload with Missing Title	User must be logged in	1. Click "Upload Content" 2. Leave Title blank 3. Select Category: Recording 4. Click Upload	Title: [blank], Category: Recording	Error message: "Title is required"	No error shown, content uploaded	FAIL
TC_04	Check Tab Navigation	User must be logged in	1. Click on "Recordings" tab	N/A	Only recording-type content is displayed	All content types are displayed	FAIL

TC- 05	Add a new student	User must be logged in as Tutor	1. Click "Add Student" 2. Enter student details 3. Click Submit	Name: Jonayed , ID: 101, Class: 8	Student appears in the student list	Student appears in the list	PASS
TC- 6	Edit student details	Student must exist in the system	1. Go to "View All Students" 2. Select a student 3. Click "Edit" 4. Change details 5. Click Save	Name: Jonayed , ID: 101, Class: 8	Student details updated in the profile	Error updating student details	FAIL
TC- 7	Remove a student	Student must exist in the system	1. Go to "View All Students" 2. Select a student 3. Click "Remove/Delete" 4. Confirm deletion	Name: Jonayed , ID: 101, Class: 8	Student removed from list	Student removed from list	PASS
TC- 8	Take attendance	Students must exist in the system	1. Go to "Take Attendance" 2. Select class 3. Mark present/absent 4. Click Submit	Class: A, Date: 2025-09-12	Attendance saved successfully	Attendance not saved	FAIL
TC- 9	View attendance list	Attendance must be recorded	1. Go to "Attendance List" 2. Select class or date	Class: A, Date: 2025-09-12	Attendance records displayed	Attendance records displayed	PASS
TC-10	Verify payment	logged in as admin, Student exists in the system.	1. Click on Add payment 2.select student 3.Enter amount 4.select payment method 5. Enter date 6. click on Save payment	student name, amount, date& time	Payment success	payment success	PASS
TC-11	Verify system prevents saving when Amount field is empty.	Student exists in the system.	1. select student 2.leave amount blank 3. enter date 4.click payment	name, date	Error message: "Amount is required."	Error message: "Amount is required."	PASS
TC-12	Verify system prevents entering negative payment amounts.	Student exists in the system.	1. select student 2.enter amount 3. enter date 4.click payment	Amount = -200	Error message: "Amount cannot be negative."	Error message: "Amount cannot be negative."	PASS
TC-13	Verify fee payment is recorded when using bKash/Nagad.	Student profile must exist, payment gateway must be integrated.	Go to Fee Management.Select student "Student A".Choose payment method "bKash".Enter amount and confirm payment.	Student A, Amount: 2000, Method: bKash.	Payment is recorded	Payment is recorded	PASS

TC -14	Verify Tutor Registration	User must be on the Tutor Registration form	Fill in all fields, Click Clear	Tutor Name: "Rahim", Age: 24, Contact: "017XXXXXXX", Email: "rahim@gmail.com"	Tutor record is saved in the database, confirmation message shown	Tutor record saved successfully	PASS
TC -15	Verify Parent Registration with SMS Option	User must be on the Parent Registration form	Enter all parent details and student id. Select "Send SMS to contact Number", Click Save	Student ID: S-101, Contact No: "018XXXXXXX"	Parent details saved, SMS notification sent	Parent details saved, SMS received	PASS
TC -16	Ensure system validates required fields properly	User must be on the Student Registration form	Enter all student details but leave E-mail blank. Click Save	Student Name: "Karim", Age: 18, Contact: "017XXXXXXX", Email: [blank]	Error message: "Email is required"	Error message displayed correctly	PASS
TC -17	Verify Clear Button Functionality in Tutor Form	User must be on the Tutor Registration form with data entered	Fill in all fields, Click Clear	Tutor Name: "Hasan", Contact: "016XXXXXXX"	All fields reset to blank	All fields cleared successfully	PASS
TC -17	Verify Clear Button Functionality in Tutor Form	User must be on the Tutor Registration form with data entered	Fill in all fields, Click Clear	Tutor Name: "Hasan", Contact: "016XXXXXXX"	All fields reset to blank	All fields cleared successfully	PASS
TC -18	Validate Contact Number Field with Invalid Input	User must be on any registration form	Enter alphabets/special characters in the Contact No field, Click Save	Contact No: "abcd1234"	Contact No: "abcd1234"	Error message displayed	PASS
TC-19	Verify Duplicate Tutor ID Handling	A tutor with ID "T-101" already exists in the database	Open the Tutor Registration form, Enter details with tutor ID "T-101", Click Save	Tutor ID: "T-101", Name: "Rahim", Contact: "017XXXXXXX"	Error message: "Tutor ID already exists"	Error message displayed correctly	PASS
TC -20	Verify SMS Not Sent When Option is Unchecked (Parent Form)	User must be on Parent Registration form	Enter all parent details and student id. Select "Send SMS to contact Number", Click Save	Student ID: "S-201", Contact No: "019XXXXXXX"	Parent details saved successfully, but no SMS sent	Parent details saved, no SMS received	PASS

7. SOFTWARE PRODUCT METRICS

Software product metrics help us **measure how good our software is** in terms of functionality, design, complexity, and maintainability.

Function-Based Metrics (Function Points)

Function points measure **the functionality delivered by the system**. We count the main components of the system:

Metric	Mentors Diary Example
External Inputs	Login, Register, Add Student, Take Attendance, Add Payment
External Outputs	Fee Report, Attendance Report, Notifications

External Inquiries View Student Progress, Check Payment Status

- Count **number of inputs, outputs, files, interfaces, and inquiries**
- Assign weights (Low, Medium, High) to each component
- Sum up to get **total function points**

These metrics check the quality of the design:

- WMC (Weighted Methods per Class): Number of methods in a class. Example: Dashboard class has 5 methods $\rightarrow$ WMC = 5
- DIT (Depth of Inheritance Tree): Levels of inheritance. Example: Tutor $\rightarrow$ PremiumTutor $\rightarrow$ AdminTutor $\rightarrow$ DIT = 3
- NOC (Number of Children): Number of subclasses. Example: Student class $\rightarrow$ 2 subclasses (UniversityStudent, CoachingStudent)
- CBO (Coupling Between Objects): How many other classes a class depends on. Example: FeeManager uses PaymentAPI & Student $\rightarrow$ CBO = 2
- LCOM (Lack of Cohesion in Methods): How related methods in a class are. Low LCOM $\rightarrow$ good cohesion

Function points help measure **how much the system does**. We do this by counting the main parts of the system:

- **Inputs:** Actions the user provides to the system.
Example: Login, Register, Add Student, Take Attendance, Add Payment
- **Outputs:** Results the system gives back to the user.
Example: Fee Report, Attendance Report, Notifications
- **Files:** Internal data stored by the system.
Example: Student Profiles, Payment Records, Content Files
- **Interfaces:** Connections with other systems or services.
Example: bKash API, Nagad API, Google Pay API
- **Inquiries:** User requests to check information.
Example: View Student Progress, Check Payment Status

These measure functions in the system:

- Average size of operations (lines of code per method)
- Complexity of operations (decision points like if/else, loops)
- Number of parameters per operation
Example: TakeAttendance() has 3 parameters (date, classID, studentList), AddPayment() has 4 parameters (studentID, amount, paymentType, date).

Maintenance Metrics

We measure **how easy it is to maintain the project over time**:

Software Maturity Index (SMI):

$SMI = \text{Total Faults Fixed} / \text{Total Faults Fixed} + \text{Remaining Faults}$

$SMI = 50 / (50 + 10) = 0.83 \rightarrow \text{High stability}$

High SMI $\rightarrow$ software is stable and can handle changes well.

8. CONCLUSION

The "Mentors Diary" project successfully addresses the widespread inefficiency of traditional management methods used by tutors and small coaching centers in Bangladesh. By creating a centralized digital platform, this project provides a comprehensive solution for student management, attendance tracking, content sharing, and fee collection, integrating localized payment systems like bKash and Nagad.

The project was executed using the

Extreme Programming (XP) model, which proved highly suitable for a small team dealing with evolving requirements from tutor feedback. A thorough design phase, documented through Use-Case, Class, Activity, and Data Flow diagrams, laid a solid foundation for the system's architecture. Development was managed with a structured

Git workflow, ensuring collaborative efficiency and a clean, traceable project history. Furthermore, the application of software product metrics provided a quantitative assessment of the software's functionality, design quality, and maintainability.

Ultimately, "Mentors Diary" stands as a feasible and creative solution tailored to its target users—tutors, students, and parents—by offering an organized, professional, and scalable tool that saves time and reduces financial losses.

1..1 Future Work

Looking ahead, the project can be expanded to further enhance its value and market reach. Potential areas for future development include:

- **Advanced Analytics:** Introduce a dedicated reporting module for tutors to analyze student performance trends and financial data over time.

- **Parent-Specific Mobile App:** Develop a separate, streamlined mobile application for parents to easily track their child's academic progress and payment status.
- **Live Class Integration:** Integrate with video conferencing platforms like Zoom or Google Meet to allow tutors to conduct online classes directly from the platform.
- **Enhanced Communication:** Implement real-time chat and announcement features to improve communication between tutors, students, and parents.
- **Gamification:** Add features like badges or points for students to encourage engagement and consistent performance.